EMBEDDED LINUX SYSTEMS

BÁO CÁO THỰC HÀNH
LAB-03

Debugging Kernel &
Phân tích Hiệu năng Ứng dụng Nhúng

Thông tin	Nội dung
Họ và tên	Nguyễn Cao Kỳ
Mã số sinh viên	SE201611
Ngày thực hiện	17/10/2026 (theo log thực nghiệm)
Môi trường	AlmaLinux 9.8 x86_64; QEMU ARMv7 Cortex-A9
Phạm vi	GDB remote, Valgrind, strace/perf, cyclictest

Báo cáo được xây dựng từ source code, log và biểu đồ của phiên LAB-03 thực tế. Các quan sát được phân biệt với suy luận kỹ thuật; không coi số liệu QEMU là benchmark của phần cứng ARM thật.

Tài liệu kèm theo: gdb_session.txt, valgrind_report.txt, valgrind_fixed.txt, strace_summary.txt, perf_report.txt, perf_stat.txt, hist_standard.txt, hist_stress.txt và latency_histogram.png.

MỤC LỤC

1. Thông tin sinh viên và môi trường

2. Phần A – Remote Debugging với GDB

3. Phần B – Memory Analysis với Valgrind

4. Phần C – Profiling với strace và perf

5. Phần D – Real-Time Latency với cyclictest

6. Trả lời câu hỏi phân tích

7. Nhận xét, khó khăn và kết luận

Phụ lục A – Toàn bộ log GDB

Phụ lục B – Trích log định lượng và đối chiếu

1. THÔNG TIN SINH VIÊN VÀ MÔI TRƯỜNG

Hạng mục	Thông tin đã xác minh / giới hạn
Sinh viên	Nguyễn Cao Kỳ – SE201611
Host	AlmaLinux 9.8, x86_64; kernel 5.14.0-687.44.1.el9_8.x86_64; 32 CPU logic
Preemption host	CONFIG_PREEMPT_DYNAMIC=y; CONFIG_PREEMPT_VOLUNTARY=y; chưa chạy kernel PREEMPT_RT
Target	QEMU system ARMv7-A Cortex-A9, máy vexpress-a9, 512 MB RAM, 2 vCPU; Linux ARM 5.15.0
Valgrind	3.26.0 (theo valgrind_report.txt)
perf	5.14.0-687.54.1.el9_8.x86_64 (theo terminal)
strace ARM / cyclictest	strace 6.1; cyclictest V 2.80; stress-ng 0.19.03; gnuplot 5.4.3
GDB host	GDB có hỗ trợ ARM và đã kết nối thành công; chưa có dòng phiên bản GDB trong bằng chứng đã cung cấp

Mục tiêu: phát hiện lỗi ứng dụng C trên ARM bằng GDB từ xa, xác định và khắc phục memory leak trên host, phân tích syscall/CPU hotspot, sau đó đo độ trễ đánh thức các thread thời gian thực của kernel host.

Các phép đo phần A và B chạy trên QEMU ARM hoặc host theo đúng tính chất từng công cụ. Phần D chạy trực tiếp trên host; phép so sánh PREEMPT_RT chỉ có tính chất lý thuyết, không có phép đo PREEMPT_RT thực tế.

2. PHẦN A – REMOTE DEBUGGING VỚI GDB

2.1. Kiến trúc và phương pháp

Ứng dụng buggy_app.c được cross-compile thành ELF ARM 32-bit, có debug symbols (-g) và tắt tối ưu (-O0). Host chạy GDB hỗ trợ ARM, target chạy gdbserver. Do bản QEMU đang dùng không hỗ trợ user-mode network/SLIRP, kết nối gỡ lỗi được triển khai qua UART ảo ttyAMA1 nối tới TCP 127.0.0.1:1234 trên host. GDB Remote Serial Protocol truyền các lệnh điều khiển giữa hai đầu.

2.2. Ba bug được xác định

Bug / địa điểm	Quan sát GDB thực tế	Nguyên nhân và cách sửa
Array out-of-bounds buggy_app.c:20,25	i=5, len=5; arr={10,20,30,40,50}; sum=150 trước khi truy cập; PC=0x1058c	Vòng lặp i <= len khiến truy cập data[5] ngoài 5 phần tử. Đổi thành i < len, kiểm tra len hợp lệ.
Use-after-free buggy_app.c:41,47	buf=0x8dda8 trước free: "Hello Embedded Linux"; khi return vẫn 0x8dda8 nhưng dữ liệu đã biến đổi.	free(buf) rồi return buf tạo con trỏ treo. Trả buffer hợp lệ, chuyển trách nhiệm free cho caller hoặc sao chép trước khi free.
Signed int overflow buggy_app.c:57	a=b=100000; giá trị toán học 10000000000; int quan sát 1410065408; PC=0x10678	Tích vượt INT_MAX=2147483647; signed overflow là undefined behavior. Ép sang int64_t trước phép nhân và in đúng kiểu.

2.3. Backtrace và thanh ghi ARM tại điểm quan sát

```
# BUG 1 (index vuot gioi han)
#0 process_data (data=0x7efffd10, len=5) at buggy_app.c:25
#1 main () at buggy_app.c:73
pc = 0x1058c; sp = 0x7efffcf8; lr = 0x1071c

# BUG 2 (con tro da free)
#0 create_message (...) at buggy_app.c:47
#1 main () at buggy_app.c:77
pc = 0x10644; buf = 0x8dda8

# BUG 3 (tran so nguyen)
#0 calculate (a=100000, b=100000) at buggy_app.c:59
#1 main () at buggy_app.c:87
pc = 0x10678; observed result = 1410065408
```

GDB đã dừng ở các breakpoint và in biến, địa chỉ, call stack và các thanh ghi pc/sp/lr. Kết thúc log là "Inferior ... exited normally": các bug được phát hiện nhờ debugger, không có bằng chứng chương trình đã crash. Toàn văn phiên GDB nằm ở Phụ lục A và trong logs/gdb_session.txt.

2.4. Đề xuất sửa lỗi

```
for (int i = 0; i < len; i++) { sum += data[i]; }

/* caller owns returned buffer */
char *msg = create_message("Hello Embedded Linux");
/* after using msg: free(msg); */

int64_t result = (int64_t)a * (int64_t)b;
```

Đây là hướng sửa kỹ thuật, không phải kết quả chạy lại ứng dụng buggy_app sau sửa. Mã nguồn nộp kèm vẫn giữ các bug cố ý để phục vụ thực hành.

3. PHẦN B – MEMORY ANALYSIS VỚI VALGRIND

3.1. Kết quả trước khi sửa

Chỉ số Valgrind	Giá trị đo
Heap in use at exit	16.900 bytes / 250 blocks
Heap operations	352 allocs; 102 frees; tổng 24.196 bytes allocated
Definitely lost	16.900 bytes / 250 blocks
Indirectly lost	0 bytes / 0 blocks
Possibly lost / still reachable	0 / 0
Error summary	3 errors from 3 contexts

3.2. Allocation stack trace và nguyên nhân

Allocation / stack trace	Dung lượng mất	Giải thích
create_record → malloc scores leaky_app.c:26; main:76	2.000 bytes, 100 blocks	100 bản ghi, mỗi mảng 5 số int × 4 bytes; delete_record() chưa free(r->scores).
create_record → malloc name leaky_app.c:22; main:76	2.100 bytes, 100 blocks	Chuỗi EmbeddedLinuxStudent có 20 ký tự + ký tự null = 21 bytes; thiếu free(r->name).
process_messages → malloc buf leaky_app.c:49; main:93	12.800 bytes, 50 blocks	50 lần cấp phát MSG_SIZE=256 bytes, thiếu free(buf) cuối mỗi vòng lặp.

Hai đối tượng name và scores được cấp phát riêng; sau khi Record cha được free, không còn con trỏ hợp lệ trỏ tới chúng. Do vậy, chúng được Valgrind phân loại là definitely lost (không phải indirectly lost). Dữ liệu trực tiếp trong log xác nhận cả ba nhóm đều "definitely lost".

3.3. Thay đổi mã và kết quả sau sửa

```
void delete_record(Record *r) {
    free(r->name);
    free(r->scores);
    free(r);
}
// Sau khi su dung buffer trong process_messages:
free(buf);
```

Binary leaky_app_fixed được kiểm tra lại bằng Valgrind, ghi nhận 352 allocs, 352 frees; "in use at exit: 0 bytes in 0 blocks", "All heap blocks were freed" và "ERROR SUMMARY: 0 errors from 0 contexts". Bản buggy gốc được giữ ở leaky_app.c; bản sửa đặt ở leaky_app_fixed.c.

3.4. Quan sát bộ nhớ trên ARM QEMU

/proc/PID/status	Trước cấp phát	Sau xử lý
PID	66	66
VmSize	776 kB	776 kB
VmRSS	4 kB	4 kB
VmData	152 kB	152 kB
VmStk	132 kB	132 kB

Bản leaky_app_arm_probe dùng SIGSTOP ở hai mốc để lấy trạng thái và /proc/PID/maps một cách lặp lại; /proc/PID/maps hiển thị vùng text, heap [0008a000–000ae000] và stack. VmRSS/VmSize không đổi trong hai snapshot. Không được coi sự không đổi này là bằng chứng không có leak: các khối cấp phát nhỏ có thể nằm trong heap đã được đặt chỗ; thông tin resident trên QEMU là ảnh chụp tức thời, trong khi Valgrind đã chứng minh memory leak bằng cách theo dõi các allocation.

Nguồn bằng chứng: valgrind_report.txt, valgrind_fixed.txt, b3_qemu_console.log, leaky_app*.c.

4. PHẦN C – PROFILING VỚI STRACE VÀ PERF

4.1. System calls của ứng dụng x86_64 (50 vòng lặp)

Thứ hạng	Syscall	Calls	Thời gian syscall
1	write	70	30,10%
2	execve	1	20,23%
3	munmap	3	12,85%
4	close	4	12,35%
5	mmap	14	9,45%

Toàn lượt strace -c có 122 syscall, 2 syscall báo lỗi và tổng 0,001206 giây thời gian được hạch toán cho syscall. Chương trình gọi fprintf() để ghi 10.000 dòng, nhưng nhờ buffer của stdio, số syscall write là 70 thay vì 10.000. Không được suy diễn tỷ lệ 30,10% thời gian syscall thành 30,10% tổng thời gian ứng dụng.

4.2. CPU hotspot từ perf record/report

Ứng dụng gốc chạy quá nhanh nên chỉ thu được 6 mẫu CPU ở lần thử thành công. Một bản perf_target_profile.c riêng tăng ITERATIONS từ 50 lên 3000 (giữ ARRAY_SIZE = 100000 và các bước xử lý còn lại); đo perf record với sự kiện cpu-clock:u 199 Hz và frame-pointer call graph, thu 395 mẫu; 0 mẫu mất. Kết quả sau đây chỉ áp dụng cho workload 3000 vòng, không gán cho bản 50 vòng.

Hạng	Symbol	Shared Object	Overhead
1	heavy_compute	perf_target_profile	51,39%
2	0x000000000007006d	libm.so.6	8,86%
3	0x0000000000070056	libm.so.6	5,82%
4	log	libm.so.6	4,30%
5	0x000000000006fffe	libm.so.6	4,05%

Các địa chỉ trong libm chưa được phân giải thành tên symbol; không tự gán chúng cho sqrt/log. heavy_compute là hotspot lớn nhất theo self-overhead. Điều này phù hợp với mã gọi sqrt và log lặp trên 100.000 phần tử.

4.3. Counter từ perf stat (bản 3000 vòng)

Counter	Kết quả
task-clock	1.979,45 ms (~0,999 CPU)
cycles:u	4.841.444.106
instructions:u	21.608.569.672
IPC	4,46 instructions/cycle
cache-misses:u	488.237
branches:u	3.001.788.192
branch-misses:u	13.534 (~0,00045%)
Elapsed / user / sys	1,980604 s / 1,979287 s / 0,001000 s

CPU time tập trung gần như hoàn toàn ở user space, đặc trưng của pha tính toán CPU-bound. Việc một số libm symbol chưa có tên và số mẫu 395 khiến bảng chỉ là ước lượng thống kê, không phải phép đo tuyệt đối từng hàm.

4.4. Profiling system calls trong QEMU ARMv7

Chỉ số	ARMv7 QEMU, 50 vòng
Kết thúc	Done.
Wall / user / sys	33,68 s / 33,62 s / 0,02 s
Số syscall / báo lỗi	100 / 1
Tổng syscall time	0,017364 s
5 syscall đứng đầu	write 76 (60,69%); munmap 2 (12,95%); brk 5 (3,86%); mmap2 2 (3,78%); statx 2 (3,75%)

Bản ARM 50 vòng cùng đầu vào cho kết quả tính toán 11433083166,0746 và checksum 99999000 giống bản host, chứng minh chạy đúng chức năng. Sự khác biệt về syscall giữa host (122) và ARM (100) đến từ ABI, trình nạp và thư viện (ARM static ELF; host dynamic ELF) cùng môi trường mô phỏng. Không so sánh 33,68 s của ARM 50 vòng trực tiếp với 1,98 s của x86_64 3000 vòng.

4.5. Bottleneck và phương án cải thiện

Trọng tâm tối ưu là công thức sqrt(x) * log(x+1) và vòng lặp lặp lại, không phải syscall write. Nên áp dụng precompute khi dữ liệu không đổi, xem xét thay đổi thuật toán/tính gần đúng có kiểm soát sai số, tối ưu độ chính xác số học và cải thiện vectorization. Phân tích sâu về NEON, loop unrolling và ước tính có điều kiện ở Câu 3 mục 6.

Nguồn: strace_summary.txt, perf_profile_info.txt, perf_report.txt, perf_stat.txt, c3_qemu_console.log, perf_target*.c.

5. PHẦN D – REAL-TIME LATENCY VỚI CYCLICTEST

5.1. Phương pháp đo

Cyclictest được chạy trên host AlmaLinux kernel PREEMPT_DYNAMIC (không phải PREEMPT_RT). Cấu hình: 4 measurement threads, SCHED_FIFO priority 80, interval 1000 µs, 10.000 loops/thread, mlockall và histogram đến 5000 µs. Cả baseline và stress sử dụng --default-system, nên cyclictest không áp dụng một số tinh chỉnh tiết kiệm năng lượng/PM QoS mặc định. Cảnh báo "not setting cpu_dma_latency" được ghi nhận, không phải lỗi thực thi.

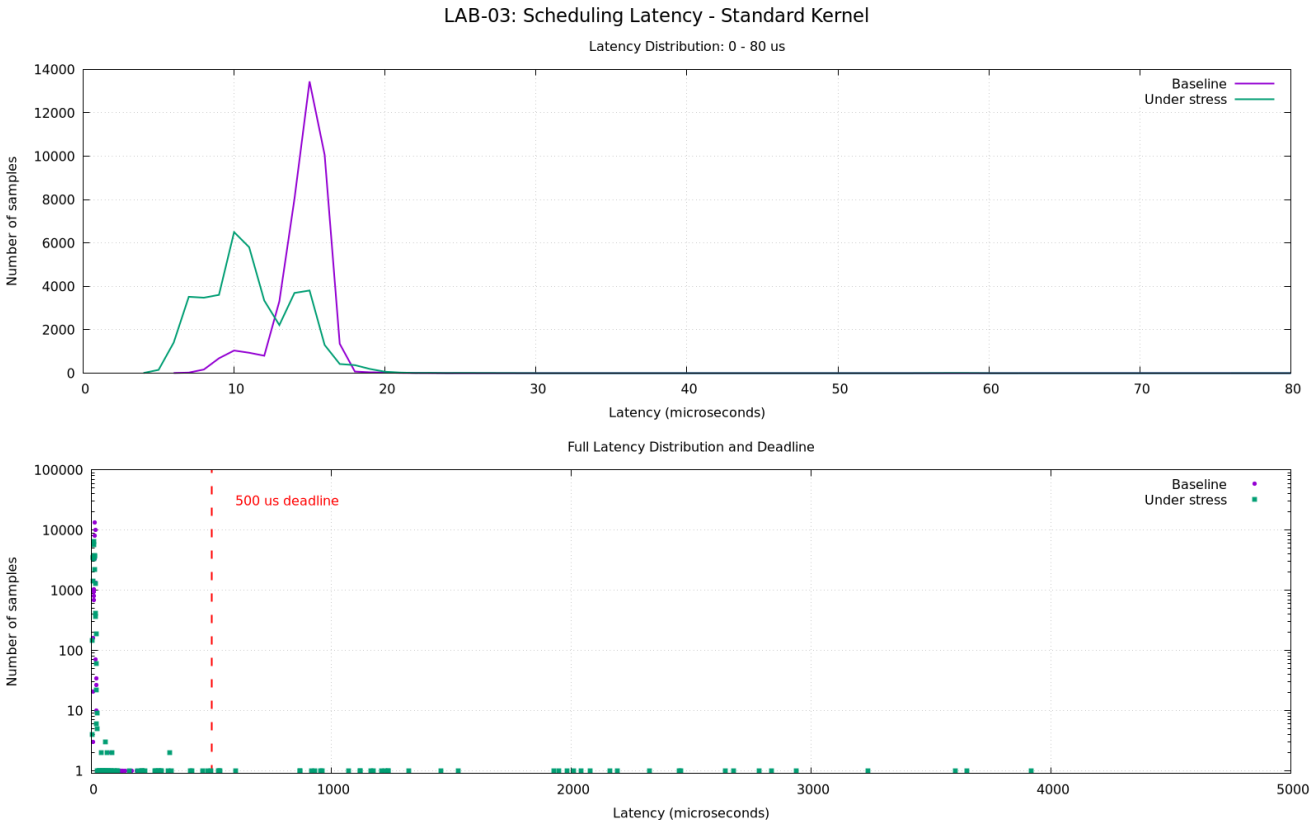
Lần chạy đầu có QEMU nền và Max=1.429 µs, nên được lưu riêng dưới tên *with_qemu*; kết quả baseline chính thức được đo lại sau khi tắt QEMU. Lần stress chạy đồng thời stress-ng 16 CPU + 4 IO + 2 VM workers trong 12,05 giây, tất cả 22 worker PASS. Log ghi rõ 64 MB/VM worker (128 MB tổng cộng), không phải 128 MB/worker như tham số đề xuất.

5.2. Min / Avg / Max theo từng thread

Điều kiện	Thread	Min (µs)	Avg (µs)	Max (µs)	Mẫu >500 µs
Baseline	0	7	14	121	0
	1	6	14	124	0
	2	8	15	142	0
	3	7	13	188	0
Stress	0	6	12	2.042	4
	1	4	10	1.121	2
	2	4	10	964	1
	3	6	19	3.919	35

Baseline: 40.000 mẫu histogram, không mẫu nào vượt 500 µs; Max quan sát 188 µs. Stress: histogram ghi nhận 39.968 mẫu, trong đó có 42 mẫu >500 µs, tức khoảng 0,105%. Thread 3 của phép đo stress có 9.968 mẫu, thiếu 32 so với số loops dự kiến; không có histogram overflow được ghi nhận. Chưa xác định được nguyên nhân chênh 32 mẫu nên không tự gán chúng cho một lỗi cụ thể.

5.3. Biểu đồ histogram từ số liệu thực



Hình 1. Histogram so sánh baseline và stress-ng; phần trên phóng to 0–80 µs, phần dưới cho thấy các đuôi phân phối và mốc deadline 500 µs. File gốc: logs/latency_histogram.png (1600×1000 pixels).

5.4. Đánh giá deadline và PREEMPT_RT

Dù Avg latency của cả hai cấu hình chỉ 10–19 µs, Max tăng từ 188 µs lên 3.919 µs khi tải stress. Với ngưỡng phản hồi <500 µs, baseline đạt trong cửa sổ đo, còn stress không đạt. Đây mới là kiểm tra scheduler wake-up latency; thời gian tính toán của tác vụ, truyền I/O và điều khiển thiết bị chưa được cộng vào deadline end-to-end.

Kernel PREEMPT_RT có thể giảm những phần không thể bị ngắt, chuyển nhiều IRQ handlers thành IRQ threads và sử dụng các primitive đồng bộ phù hợp RT, nhờ đó giảm worst-case wake-up latency. Tuy nhiên, tài liệu này KHÔNG có phép đo kernel PREEMPT_RT thực tế: không tuyên bố mức cải thiện định lượng so với Standard từ kết quả nêu trên.

Nguồn: hist_standard.txt, hist_stress.txt, d_latency_summary.txt, d_stressng.txt, d_standard_environment.txt, d_host_kernel_info.txt.

6. TRẢ LỜI CÂU HỎI PHÂN TÍCH

Câu 1. Chuỗi sự kiện khi GDB gửi lệnh next; giao thức

Khi người dùng nhập next ở GDB trên host, GDB xử lý logic step-over, gửi lệnh điều khiển tới gdbserver qua GDB Remote Serial Protocol (RSP). Trong cấu hình thực nghiệm, gói RSP đi trên kết nối TCP localhost:1234 rồi được QEMU chuyển thành dữ liệu serial tới ttyAMA1. gdbserver điều khiển process ARM bằng cơ chế debug của Linux (như ptrace), đặt breakpoint tạm nếu cần để step-over, tiếp tục cho process chạy. Khi process dừng bởi breakpoint/sự kiện, gdbserver trả stop reply và trạng thái thanh ghi cho GDB; GDB đọc bộ nhớ, biểu thức và hiển thị dòng source tương ứng với DWARF debug symbols.

Trên host x86_64, bản gdb-multiarch hoặc GDB build có hỗ trợ ARM phải hiểu register set, breakpoint và ABI ARM. GDB chỉ biên dịch cho x86_64 chưa chắc có ARM backend. Thực tế log cho thấy kết nối thành công tới 127.0.0.1:1234, hiển thị pc/sp/lr và source buggy_app.c; đây là bằng chứng GDB host có khả năng debug ARM.

Câu 2. Definitely lost và indirectly lost; các lỗi khác Valgrind phát hiện

Definitely lost: Valgrind không tìm thấy con trỏ nào còn trỏ hợp lệ tới vùng cấp phát khi kết thúc process. Indirectly lost: vùng cấp phát chỉ có thể đi qua một vùng cấp phát khác đã bị mất (ví dụ phần tử con của một struct cha cũng bị leak). Trong leaky_app.c, r được free còn r->name và r->scores không free; sau khi r bị giải phóng không còn chuỗi con trỏ sở hữu nào có thể truy cập hai allocations, nên cả name và scores được ghi definitely lost. Log có 2.100 + 2.000 + 12.800 = 16.900 bytes definitely lost; indirectly lost bằng 0.

Ngoài leak, Memcheck còn phát hiện invalid read/write (kể cả out-of-bounds), use-after-free/double free, sử dụng giá trị chưa khởi tạo và lỗi quản lý heap như mismatched malloc/free (tùy tình huống). Kết luận dựa trên log Valgrind của chương trình này, không khẳng định mọi lỗi nêu ra đều đã xuất hiện.

Câu 3. Ít nhất hai phương án tối ưu heavy_compute

Phương án 1 – loại bỏ phép tính lặp có đầu vào bất biến: darr không thay đổi trong 3000 vòng. Có thể tính một lần heavy_compute(darr, N) và nhân kết quả với ITERATIONS; giảm từ 3000 lần quét mảng còn một lần. Về số phép toán lý tưởng, phần tính toán có thể giảm gần 3000 lần; tốc độ end-to-end thấp hơn vì overhead khác, và cần kiểm tra sai số do thay đổi thứ tự cộng số thực. Đây là ước lượng thuật toán, chưa có benchmark sau tối ưu.

Phương án 2 – SIMD/NEON, loop unrolling và tính gần đúng: với phần tính toán độc lập giữa các phần tử, có thể unroll 2–4 phần tử để tăng ILP, đồng thời đo khả năng vector hóa. Tuy nhiên ARMv7 NEON không thực thi vector double-precision sqrt/log như x86 hiện đại; muốn dùng vector float32 hoặc xấp xỉ đa thức/lookup phải chấp nhận và đo sai số, độ chính xác và xử lý ngoại lệ. Ví dụ GIẢ ĐỊNH kernel tính toán chiếm 80% thời gian, tăng tốc chính phần này 2× cho speedup tổng $1/(0{,}2+0{,}8/2)\approx 1{,}67\times$ theo Amdahl. Đây chỉ là ví dụ ước lượng có điều kiện, không phải hiệu quả đo được.

Phương án 3 – tiền xử lý mảng và locality: nếu mảng dùng nhiều lần với dữ liệu không đổi, lưu sẵn f(x)=sqrt(x)*log(x+1) hoặc kết quả reduction. Nếu dữ liệu thay đổi, profile trước/sau, đo cache misses và mức sử dụng vector instruction; không thể hứa phần trăm tăng tốc khi chưa có thử

nghiệm. Bằng chứng gốc: heavy_compute chiếm 51,39% self-overhead, libm chiếm thêm nhiều mẫu.

Câu 4. Vì sao latency spike, PREEMPT_RT giải quyết thế nào?

Một là ngắt cứng và xử lý interrupt quá lâu: thread real-time không thể chạy cho tới khi IRQ hoàn thành; PREEMPT_RT chuyển phần lớn xử lý IRQ thành thread có priority để bộ lập lịch điều phối tốt hơn, ngoại trừ phần IRQ tối thiểu bắt buộc chạy non-threaded. Hai là các đoạn kernel giữ spinlock hoặc tắt preemption/interrupt khiến thread ưu tiên cao phải chờ; PREEMPT_RT thay nhiều spinlock thường bằng sleeping rtmutex hỗ trợ priority inheritance, giảm priority inversion và vùng non-preemptible. Ba là page fault, reclaim hoặc tranh chấp bộ nhớ gây trễ: mlockall giúp tránh fault cho bộ nhớ đã lock; PREEMPT_RT hạn chế một số critical section nhưng không triệt tiêu mọi fault, reclaim, SMI hay tác động driver.

Kết quả thực nghiệm hỗ trợ ý nghĩa worst case: baseline Max=188 µs, khi stress Max=3.919 µs và có 42 mẫu >500 µs. Hai lần chạy không cho biết chính xác từng spike do IRQ, lock hay memory pressure; cần thêm ftrace/trace-cmd/perf sched để truy nguồn chính xác. Kết quả PREEMPT_RT chỉ suy luận theo cơ chế thiết kế, không phải dữ liệu thử nghiệm.

Câu 5. Chọn công cụ theo loại vấn đề

Tình huống	Công cụ chính	Vì sao / phối hợp
(a) Crash ngẫu nhiên sau vài giờ	GDB + core dump	Phân tích stack, biến và PC ở nơi crash; có thể chạy Valgrind bổ sung nếu nghi memory corruption.
(b) Càng chạy càng chậm	Valgrind; /proc/PID/status	Phân tích memory leak và tăng RSS; nếu không do leak, dùng strace/perf để kiểm tra I/O, CPU.
(c) Đôi khi bỏ lỡ deadline	cyclictest	Đo Max latency/histogram dưới tải, xác định mức độ vi phạm deadline; phối hợp trace IRQ/scheduler.
(d) Một hàm làm hệ thống chậm	perf	Ghi CPU samples, call graph và hotspots; dùng strace nếu suspect chờ I/O syscall.

7. NHẬN XÉT, KHÓ KHĂN VÀ KẾT LUẬN

7.1. Khó khăn đã gặp và hướng xử lý

Khó khăn cụ thể	Cách giải quyết thực hiện
QEMU bản build không hỗ trợ SLIRP/-net user nên không dùng TCP hostfwd trực tiếp.	Dùng gdbserver qua ttyAMA1 và UART-to-TCP bridge, GDB kết nối 127.0.0.1:1234.
perf chạy dưới sudo tạo xung đột ghi file /tmp; chương trình kết thúc sớm với fopen Permission denied.	Chạy perf với cùng user nguyenky; tách binary profiling, mở rộng số vòng lặp để đạt 395 mẫu.
Bản x86_64 strace không thể chạy trong ARM QEMU; rootfs chưa có ARM strace.	Lấy binary strace armhf từ gói Debian Bookworm qua container, kiểm tra ELF và dependency, cập nhật initramfs.
cyclictest cài xong nhưng không có quyền thực thi với user thường; baseline đầu còn QEMU đang chạy.	Xác minh RPM bằng sudo, chạy cyclictest sudo; lưu riêng baseline có QEMU, tắt QEMU và đo lại baseline sạch.

7.2. Kết luận cuối cùng

Phần A xác định ba lỗi trên ARM bằng GDB remote; Phần B phát hiện và khắc phục 16.900 bytes definitely lost; Phần C xác định heavy_compute và các symbol libm là vùng CPU hotspot trên workload profiling, đồng thời so sánh system calls host/ARM; Phần D thu histogram định lượng cho Standard Kernel. Kết quả stress có 42 lần latency >500 µs và Max=3.919 µs, vì vậy chưa thể bảo đảm yêu cầu điều khiển robot phản hồi <500 µs trong môi trường tải đã thử.

Hướng tiếp tục: đo lặp nhiều lần với điều kiện phần cứng và CPU governor kiểm soát, thử nghiệm PREEMPT_RT thật, kết hợp tracing scheduler/IRQ và mô hình deadline end-to-end. Các kết luận về hiệu quả PREEMPT_RT hoặc tối ưu SIMD cần benchmark mới để xác minh.

7.3. Danh mục chứng cứ

Nộp kèm source ứng dụng, binary ARM/x86_64, initramfs_lab3.cpio.gz, toàn bộ logs/gdb_session.txt, logs/valgrind_report.txt, logs/valgrind_fixed.txt, logs/strace_summary.txt, logs/perf_report.txt, logs/perf_stat.txt, logs/hist_standard.txt và logs/latency_histogram.png. Các logs bổ sung: b3_qemu_console.log, c3_qemu_console.log, hist_stress.txt, d_latency_summary.txt, d_stressng.txt.

Tài liệu nền: đề bài LAB03_EmbeddedLinux.docx; GDB Remote Serial Protocol documentation; Valgrind Memcheck manual; Linux perf tools documentation; cyclictest man page; tài liệu Linux PREEMPT_RT. Các kết quả định lượng trong báo cáo đều từ bộ logs do sinh viên cung cấp.

PHỤ LỤC A – TOÀN BỘ GDB SESSION LOG

Bản ghi nguyên văn được lưu trong logs/gdb_session.txt. Log dưới đây bao gồm kết nối, breakpoint, print biến, backtrace, registers và kết thúc process.

```
Remote debugging using 127.0.0.1:1234
0x00010420 in _start ()
Breakpoint 1 at 0x106b8: file buggy_app.c, line 67.
Num     Type           Disp Enb Address    What
1       breakpoint     keep y   0x000106b8 in main at buggy_app.c:67
Breakpoint 2 at 0x10578: file buggy_app.c, line 18.
Continuing.

Breakpoint 1, main () at buggy_app.c:67
67        printf("=== LAB03 BUGGY APP ===\n");
Continuing.

Breakpoint 2, process_data (data=0x7efffd10, len=5) at buggy_app.c:18
18        int sum = 0;
data = 0x7efffd10
len = 5
$1 = 5
$2 = {10, 20, 30, 40, 50}
#0  process_data (data=0x7efffd10, len=5) at buggy_app.c:18
#1  0x0001071c in main () at buggy_app.c:73
13     */
14
15    /* BUG 1: Loop executes one iteration too many. */
16    void process_data(int *data, int len)
17    {
18        int sum = 0;
19
20        for (int i = 0; i <= len; i++) {
21            /*
22             * BUG: Valid indices are 0 to len-1.
Breakpoint 3 at 0x1058c: file buggy_app.c, line 25.
Continuing.

Breakpoint 3, process_data (data=0x7efffd10, len=5) at buggy_app.c:25
25            sum += data[i];
$3 = 5
$4 = 5
$5 = 0
$6 = 1
i = 5
sum = 150
#0  process_data (data=0x7efffd10, len=5) at buggy_app.c:25
#1  0x0001071c in main () at buggy_app.c:73
pc             0x1058c             0x1058c <process_data+40>
sp             0x7efffcf8          0x7efffcf8

lr             0x1071c             67356
$7 = {10, 20, 30, 40, 50}
Breakpoint 4 at 0x1063c: file buggy_app.c, line 41.
Breakpoint 5 at 0x10644: file buggy_app.c, line 47.
Continuing.

Breakpoint 4, create_message (text=0x6ee18 "Hello Embedded Linux") at buggy_app.c:41
41        free(buf);
$8 = 0x8dda8 "Hello Embedded Linux"
0x8dda8:    "Hello Embedded Linux"
#0  create_message (text=0x6ee18 "Hello Embedded Linux") at buggy_app.c:41
#1  0x0001073c in main () at buggy_app.c:77
35
36        if (buf == NULL)
37            return NULL;
38
39        strcpy(buf, text);
40
41        free(buf);
42
43        /*
44         * BUG: buf no longer points to valid
```

```
45         * allocated memory after free().
46         */
47        return buf;
48    }
Continuing.

Breakpoint 5, create_message (text=0x6ee18 "Hello Embedded Linux") at buggy_app.c:47
47        return buf;
$9 = 0x8dda8 "\215"
buf = 0x8dda8 "\215"
#0  create_message (text=0x6ee18 "Hello Embedded Linux") at buggy_app.c:47
#1  0x0001073c in main () at buggy_app.c:77
pc             0x10644             0x10644 <create_message+88>
sp             0x7efffcf8          0x7efffcf8
lr             0x2c1fc             180732
Breakpoint 6 at 0x10668: file buggy_app.c, line 57.
Continuing.

Breakpoint 6, calculate (a=100000, b=100000) at buggy_app.c:57
57        int result = a * b;
a = 100000
b = 100000
$10 = 100000

$11 = 100000
$12 = 10000000000
$13 = 4
$14 = 2147483647
59        printf("calculate(%d, %d) = %d\n",
$15 = 1410065408
#0  calculate (a=100000, b=100000) at buggy_app.c:59
#1  0x0001077c in main () at buggy_app.c:87
pc             0x10678             0x10678 <calculate+36>
sp             0x7efffcf8          0x7efffcf8
lr             0x1077c             67452
54         * BUG: 100000 * 100000 exceeds INT_MAX.
55         * Signed integer overflow is undefined behavior.
56         */
57        int result = a * b;
58
59        printf("calculate(%d, %d) = %d\n",
60               a, b, result);
61
62        return result;
63    }
Continuing.
[Inferior 1 (process 71) exited normally]
```

PHỤ LỤC B – TRÍCH LOG ĐỐI CHIẾU

B1. Trích toàn văn leak summary trước và sau sửa

```
==21779== Memcheck, a memory error detector
==21779== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==21779== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==21779== Command: /home/nguyenky/embedded_lab3/apps/leaky_app
==21779== Parent PID: 21770
==21779==
==21779==
==21779== HEAP SUMMARY:
==21779==     in use at exit: 16,900 bytes in 250 blocks
==21779==   total heap usage: 352 allocs, 102 frees, 24,196 bytes allocated
==21779==
==21779== 2,000 bytes in 100 blocks are definitely lost in loss record 1 of 3
==21779==    at 0x484782F: malloc (vg_replace_malloc.c:447)
==21779==    by 0x4011F9: create_record (leaky_app.c:26)
==21779==    by 0x401356: main (leaky_app.c:76)
==21779==
==21779== 2,100 bytes in 100 blocks are definitely lost in loss record 2 of 3
==21779==    at 0x484782F: malloc (vg_replace_malloc.c:447)
==21779==    by 0x4011C6: create_record (leaky_app.c:22)
==21779==    by 0x401356: main (leaky_app.c:76)
==21779==
==21779== 12,800 bytes in 50 blocks are definitely lost in loss record 3 of 3
==21779==    at 0x484782F: malloc (vg_replace_malloc.c:447)
==21779==    by 0x401281: process_messages (leaky_app.c:49)
==21779==    by 0x4013BF: main (leaky_app.c:93)
==21779==
==21779== LEAK SUMMARY:
==21779==    definitely lost: 16,900 bytes in 250 blocks
==21779==    indirectly lost: 0 bytes in 0 blocks
==21779==      possibly lost: 0 bytes in 0 blocks
==21779==    still reachable: 0 bytes in 0 blocks
==21779==         suppressed: 0 bytes in 0 blocks
==21779==
==21779== For lists of detected and suppressed errors, rerun with: -s
==21779== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
```

```
==22233== Memcheck, a memory error detector
==22233== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==22233== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==22233== Command: /home/nguyenky/embedded_lab3/apps/leaky_app_fixed
==22233== Parent PID: 22221
==22233==
==22233==
==22233== HEAP SUMMARY:
==22233==     in use at exit: 0 bytes in 0 blocks
==22233==   total heap usage: 352 allocs, 352 frees, 24,196 bytes allocated
==22233==
==22233== All heap blocks were freed -- no leaks are possible
==22233==
==22233== For lists of detected and suppressed errors, rerun with: -s
==22233== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

B2. Tóm tắt perf record/stat

```
Samples: 395 of event cpu-clock:u; Total Lost Samples: 0
heavy_compute: 51.39% self-overhead
Task clock: 1979.45 ms; cycles: 4841444106
Instructions: 21608569672; IPC: 4.46
Elapsed: 1.980603788 s
```

B3. Kiểm tra bảo toàn bộ dữ liệu

Toàn bộ source/log gốc không bị sửa nội dung khi tạo PDF. Các đoạn trích định lượng luôn ghi đơn vị, số vòng và môi trường thực thi. Gói bài nộp cuối cùng cần được tạo trên AlmaLinux để bao gồm binary và initramfs vốn không có trong gói dữ liệu dùng soạn báo cáo.